Virtual Functions

e Ereditarietà Virtuale

Teoria completa

Appunti di Ingegneria del Software

Basato sulle slide del corso (*Virtual Functions and Late Bindings*), su Wikipedia (*Virtual function*, *Virtual inheritance*) e su *Thinking in C++* (Bruce Eckel)

Late binding, overriding, VFT/VFTP, dominanza,
diamond problem, basi virtuali

Ogni esempio di codice di questo documento è stato compilato ed eseguito davvero (cartella `prove/`*): dove il comportamento reale si è rivelato diverso da quello "da manuale", è segnalato esplicitamente.*

Anno 3

Contents

Chapter 1 Perché servono le funzioni virtuali — Page 3
1.1 Il problema: comportamento deciso il più tardi possibile 3
1.2 Overriding: le condizioni 3
1.3 Le tre possibilità del dispatch 4

Chapter 2 Virtual functions in Java e in C++ — Page 5
2.1 Java: tutto è virtual di default 5
2.2 C++: virtual va dichiarato esplicitamente 5

Chapter 3 Come si implementa il late binding — Page 7
3.1 Early binding vs late binding 7
3.2 VFT e VFTP: l'implementazione 7

Chapter 4 Virtual vs. non-virtual: l'efficienza — Page 9

Chapter 5 L'esercizio del corso — Page 10

Chapter 6 Ereditarietà virtuale — Page 11
6.1 Il problema: sotto-oggetti duplicati 11
6.2 Chi costruisce la base virtuale 11
6.3 Dominanza 12
6.4 Layout e costo in memoria 12
6.5 Metodi virtuali puri nel diamante 12
6.6 Quando usarla 13

Chapter 7 Oltre le slide: approfondimenti da Wikipedia — Page 14
7.1 Il corner case: una funzione virtuale pura può avere un corpo 14
7.2 Il consiglio più citato: mai chiamare virtual da ctor/dtor 15
7.3 Argomenti collegati (See also) 15

Chapter — Overriding in Java — Page 16
8.1 Overriding automatico, e l'annotazione `@Override` 16
8.2 Cosa non può essere overridden 16
8.3 super: chiamare l'implementazione della base 17
8.4 I campi NON sono polimorfici 17
8.5 Return type covariante 17
8.6 La chiamata a un metodo overridabile dal costruttore 17

Chapter — Overriding in Python — Page 18
9.1 Nessuna keyword, perché non serve 18
9.2 super() e l'MRO: ereditarietà multipla cooperativa 18
9.3 Name mangling: l'equivalente di un metodo non virtuale 19
9.4 Metodi "abstract": @abstractmethod da solo non basta 19
9.5 Il costruttore e il dispatch 19

Chapter — Polimorfismo manuale in C — Page 20
10.1 La traduzione letterale di VFT e VFTP 20
10.2 Chi inizializza il vptr: qui nessuno, se non tu 21
10.3 Esempi reali: non è un esercizio accademico 21

Chapter — Riferimenti e link aggiuntivi — Page 22
11.1 Dalla voce "Virtual function" 22
11.2 Dalla voce "Virtual inheritance" 22
11.3 Dalla voce "Method overriding" 22
11.4 Thinking in C++ (Bruce Eckel) — libero, due volumi 22

Chapter — Fonti — Page 23

Chapter 1

Perché servono le funzioni virtuali

1.1 Il problema: comportamento deciso il più tardi possibile

Una delle idee centrali della programmazione a oggetti è che il comportamento di un oggetto venga determinato **al momento più tardo possibile**, se possibile a runtime. Nei linguaggi imperativi convenzionali, questo era di solito possibile solo tramite istruzioni `switch` o `if`. Le cosiddette **funzioni virtuali** sono una soluzione elegante e flessibile a questo problema: implementano un meccanismo chiamato **late binding**.
Fonte: slide ExtraLectureOnVF*, pag. 3.*

Definition 1.1.1: Funzione virtuale

Un metodo (non statico) di una classe base che può essere "ridefinito" nelle classi derivate. Questa ridefinizione si chiama **overriding**.

Fonte: slide ExtraLectureOnVF*, pag. 5.*

1.2 Overriding: le condizioni

L'overriding avviene solo se:

- il metodo sovrascritto (nella base) è **virtual**;
- il metodo che sovrascrive (nella derivata) ha la **stessa signature** e lo **stesso return type** del metodo sovrascritto.

L'overriding può essere impedito in certi linguaggi di programmazione.
Fonte: slide ExtraLectureOnVF*, pag. 6.*

Definition 1.2.1: Signature vs. prototype

La **signature** di una funzione include: nome, numero di parametri formali, tipi dei parametri formali. Il **return type non è incluso** nella signature. Signature + return type insieme si chiamano il **prototype** della funzione.

Fonte: slide ExtraLectureOnVF*, pag. 7.*

Proposition 1.2.1 Conseguenze pratiche delle regole di overriding

- i metodi **static non possono essere overridden**;
- stessa signature ma return type diverso → **errore di compilazione** (il C++ ammette solo lievi variazioni covarianti del return type);
- in **Java**, stesso nome ma signature diversa → **l'overriding non avviene** (overloading, non override);

- in **C++**, stesso nome ma signature diversa → il metodo con quel nome nella base diventa **invisibile** (hiding) nella derivata. L'overloading può avvenire o meno a seconda del linguaggio.

Fonte: slide ExtraLectureOnVF*, pag. 8-9.*
Verificato in `prove/03_name_hiding_overload.cpp` *(compilato ed eseguito).*

1.3 Le tre possibilità del dispatch

Quando una funzione virtuale `f` viene invocata su un oggetto `o` di classe `D` derivata da `B`, ci sono tre possibilità: (1) `f` è definita solo in `D` → si chiama `D.f`; (2) `f` è definita solo in `B` → si chiama `B.f` (ereditata); (3) `f` è overridden in `D` → si chiama la versione overridden `D.f`.

Proposition 1.3.1 La regola fondamentale

Si usa sempre la **classe reale** dell'oggetto, non la classe della variabile/riferimento/puntatore usato per accedervi.

Fonte: slide ExtraLectureOnVF*, pag. 11.*
Verificato in `prove/01_dispatch_realtype.cpp` *(compilato ed eseguito).*
Il vero vantaggio delle funzioni virtuali emerge quando si manipolano oggetti tramite **riferimenti o puntatori alla loro classe base**: solo in questo caso il comportamento "virtuale" diventa visibile all'utente.
Fonte: slide ExtraLectureOnVF*, pag. 12.*

Chapter 2

Virtual functions in Java e in C++

2.1 Java: tutto è virtual di default

In Java **tutti** i metodi non statici sono virtual: l'overriding è sempre un'opzione. Per impedire l'overriding (e l'hiding) di un metodo si usa la keyword `final`; dichiarare `final` un'intera classe ne impedisce l'estensione (non può essere usata come base per ulteriore derivazione).
Fonte: slide ExtraLectureOnVF, *pag. 16.*

Example 2.1.1 (L'esempio Animal/Dog/Cat delle slide)

```
public class Animal {
    public void voice() { System.out.println("???"); }
}
public class Dog extends Animal {
    public void voice() { System.out.println("bau"); }
}
public class Cat extends Animal {
    public void voice() { System.out.println("miao"); }
}
public class VirtualF {
    public static void main(String argv[]) {
        Animal a = new Dog();
        Animal b = new Cat();
        a.voice();   // both a and b are references to "Animals"
        b.voice();
    }
}
```

Output: `bau` (`a` fa riferimento a un `Dog`), `miao` (`b` a un `Cat`) — il dispatch usa sempre la classe reale.

Fonte: slide ExtraLectureOnVF, *pag. 25.*
Verificato in `prove/10_VirtualFunctionsInJava.java` *(compilato ed eseguito).*

2.2 C++: virtual va dichiarato esplicitamente

In C++ un metodo deve essere dichiarato `virtual` esplicitamente. Un metodo **non** dichiarato `virtual` **non può essere overridden**: può solo essere **nascosto** da una nuova definizione. Non esiste, in C++, una keyword equivalente a `final` di Java: non è possibile impedire l'hiding di un metodo. La derivazione di classe è invece sempre possibile.
Fonte: slide ExtraLectureOnVF, *pag. 18.*

```
class B {
public:
    virtual int compute(int x) { return theInt + x; }
};
class D : public B {
public:
```

```
    int compute(int x) { return x * theInt; }   // 'virtual' puo' essere omesso
};
```

Fonte: slide ExtraLectureOnVF*, pag. 20.*
Verificato in ***prove/02_hiding_non_virtual.cpp*** *(compilato ed eseguito).*

Wrong Concept 2.1: Se scrivo una funzione con lo stesso nome in una derivata, sto sempre facen

No: se la funzione della base **non** è `virtual`, quella che scrivi nella derivata **nasconde** la versione della base, non la sovrascrive. Late binding non viene mai eseguito per quel metodo (esempio 02): chiamandolo tramite un puntatore/riferimento alla base, si esegue sempre la versione della base, indipendentemente dalla classe reale dell'oggetto.

Chapter 3

Come si implementa il late binding

3.1 Early binding vs late binding

Definition 3.1.1: Early binding (compile-time binding)

Nei linguaggi imperativi non-OO (C, Pascal...), quando il compilatore incontra una chiamata a funzione scrive codice macchina che esegue una chiamata a un **indirizzo ben definito** — l'indirizzo di memoria dove inizia il codice macchina della funzione. Il compilatore "lega" il punto di chiamata al codice giusto già a **compile time**. È il comportamento standard per funzioni e metodi C++ non virtual.

Fonte: slide ExtraLectureOnVF*, pag. 29-30.*

Definition 3.1.2: Late binding (run-time binding)

In presenza di funzioni virtuali, il binding **non può** essere fatto a compile time: il compilatore sa quale funzione è virtual, ma non può sempre dedurre quale codice eseguire guardando solo la chiamata. Scrive invece codice macchina che deduce l'indirizzo corretto della chiamata **a runtime**.

Fonte: slide ExtraLectureOnVF*, pag. 31.*

3.2 VFT e VFTP: l'implementazione

Ogni classe con funzioni virtuali ha una tabella associata, chiamata **Virtual Function Table (VFT)**, contenente i puntatori al codice di ciascuna funzione virtuale della classe. Ogni oggetto di una classe con funzioni virtuali porta un puntatore, **nascosto all'utente**, alla VFT della propria classe: il **Virtual Function Table Pointer (VFTP)**.
Fonte: slide ExtraLectureOnVF*, pag. 33.*

Note:

In questo contesto "pointer" significa sia un puntatore C++ sia un riferimento Java: il meccanismo è concettualmente lo stesso nei due linguaggi, anche se Java lo nasconde completamente.

Proposition 3.2.1 I cinque passi del dispatch a runtime

Per completare una chiamata late-bound:

1. dal puntatore/riferimento si raggiunge l'**oggetto**;
2. si cerca la **VFTP** nell'oggetto;
3. tramite la VFTP si trova la **VFT della classe reale** dell'oggetto;
4. si cerca nella VFT l'**entry giusta** (es. VFP2 per la funzione VF2);
5. tramite il puntatore in quella entry si raggiunge ed **esegue** il codice corretto.

Fonte: slide ExtraLectureOnVF*, pag. 32,34-35.*

Questa è esattamente la traduzione in C che si può scrivere a mano (struct con un puntatore a una tabella di puntatori a funzione): vedi il capitolo sui template per il parallelo con la genericità in C.

Chapter 4

Virtual vs. non-virtual: l'efficienza

Se la "virtualità" è così comoda, perché non usare sempre metodi virtual? Java prende questo approccio, vincolando l'utente a usare solo metodi virtual. Il C++ dà più libertà, permettendo anche metodi non virtual — ma perché?
Fonte: slide ExtraLectureOnVF*, pag. 36.*

Proposition 4.0.1 La risposta: efficienza

Il late binding è un meccanismo relativamente costoso in termini di tempo di esecuzione (in misura diversa secondo l'implementazione usata). Se un metodo viene chiamato molto frequentemente ed è relativamente semplice, l'overhead può diventare un collo di bottiglia.

Fonte: slide ExtraLectureOnVF*, pag. 37.*
In pratica, però, ci sono pochissimi punti (se non nessuno) in un programma medio dove questi problemi di efficienza sono determinanti; ma in **sistemi real-time**, **routine matematiche core** o **programmazione di sistema di basso livello** può essere il caso.
Fonte: slide ExtraLectureOnVF*, pag. 38.*

Note:

Questo è lo stesso trade-off discusso nella sezione trasversale del fascicolo di domande d'esame: C++ non-virtual-by-default segue lo *zero-overhead principle*; Java virtual-by-default recupera il costo con la devirtualizzazione JIT (Class Hierarchy Analysis + inline cache).

Chapter 5

L'esercizio del corso

Le slide concludono con un esercizio completo su una gerarchia a sei classi (`A`, `B`, `C`, `D`, `E`, `G`), che mescola deliberatamente metodi virtual e non-virtual, override e hiding, per costringere a ragionare riga per riga.
Fonte: slide ExtraLectureOnVF*, pag. 40-45.*

Example 5.0.1 (Le classi della gerarchia (estratto))

```
class A {
public:
    virtual void f() { prn("A::f()"); }
             void g() { prn("A::g()"); }
    virtual void h() { prn("A::h()"); }
             void i() { prn("A::i()"); }
};
class D : public A {
public:
    virtual void h() { prn("D::h()"); }
             void m() { prn("D::m()"); h(); }
    virtual void g() { prn("D::g()"); }
};
class G : public E {   // E deriva da D
public:
    virtual void f(int i) { prn("G::f(int)"); }
};
```

Wrong Concept 5.1: Basta guardare quale metodo è virtual per sapere cosa stampa una chiamat

No: bisogna anche controllare, per ogni chiamata, il **tipo statico** usato per farla (variabile, puntatore, riferimento). Tre righe di questo esercizio non compilano affatto, per **name hiding**: `G` dichiara `f(int)`, che nasconde ogni `f()` ereditato — quindi `aG.f()` (senza argomenti) non trova nulla da chiamare, anche se una `f()` esiste più in alto nella gerarchia.

L'esercizio completo, da fare tu stesso prima di guardare la soluzione, è in `prove/es_OO_slide_ABCDEG.cpp` (soluzione verificata in `prove/soluzioni.md`).

Chapter 6

Ereditarietà virtuale

Le slide del corso non trattano l'ereditarietà virtuale: questo intero capitolo si appoggia alla voce Wikipedia *Virtual inheritance* e a *Thinking in C++* (Vol. 2, cap. 9, *Multiple Inheritance*).
Fonte: Wikipedia, Virtual inheritance.

6.1 Il problema: sotto-oggetti duplicati

Quando due classi `B` e `C` derivano da `A`, e una classe `D` deriva da entrambe `B` e `C`, l'oggetto `D` contiene **due copie** delle variabili membro di `A` — una via `B`, una via `C` — accessibili indipendentemente tramite scope resolution. Questo è il **diamond problem**.
Fonte: Wikipedia, Virtual inheritance.
Fonte: Thinking in C++, *Vol. 2, cap. 9, "Multiple Inheritance".*

Definition 6.1.1: Ereditarietà virtuale

Tecnica C++ che garantisce che **una sola copia** delle variabili membro di una classe base venga ereditata dalle classi derivate "nipote". Si dichiara con `virtual` nella lista delle basi: `class Derived : virtual public Base { ... };`

Fonte: Wikipedia, Virtual inheritance.
Fonte: Thinking in C++, *Vol. 2, cap. 9.*

6.2 Chi costruisce la base virtuale

Proposition 6.2.1 La regola dell'inizializzazione

La base virtuale viene inizializzata **sempre** dal costruttore della classe **most derived** — mai da una classe intermedia, anche se quella intermedia scrive un mem-initializer per la base virtuale: se la classe intermedia non è quella davvero istanziata, quel mem-initializer viene **ignorato**. Questo impedisce che più classi intermedie inizializzino la base virtuale più volte.

Fonte: Thinking in C++, *Vol. 2, cap. 9, sezione "Virtual base classes".*
Verificato in `prove/08_virtual_base_ctor_order.cpp` *(compilato ed eseguito).*

Example 6.2.1 (Chi vince, tra i mem-initializer scritti a ogni livello)

```
struct A { A(const char* who) { std::cout << who << " "; } };
struct B : virtual A { B() : A("da B") {} };
struct C : virtual A { C() : A("da C") {} };
struct D : B, C  { D() : A("da D"), B(), C() {} };
struct E : D     { E() : A("da E"), D()     {} };
int main() { E e; }
```

Output verificato: `A("da E") B() C() D() E()` — solo il mem-initializer di `E` (la classe most-derived) conta;

`"da B"`, `"da C"`, `"da D"` vengono tutti ignorati in questa costruzione. Costruendo un `D` da solo, invece, vince `"da D"`.

6.3 Dominanza

Definition 6.3.1: Dominanza

Con basi virtuali esiste un **solo** sotto-oggetto condiviso: se solo una delle due classi intermedie fa l'override di un metodo virtuale della base condivisa, quell'override **domina** sull'implementazione originale, e diventa l'unico *final overrider*: nessuna ambiguità.

Verificato in `prove/06_diamond_dominance.cpp` (compilato ed eseguito).

Wrong Concept 6.1: Senza virtual, il diamante darebbe semplicemente lo stesso risultato ma più

No: senza `virtual` nelle basi, esistono **due** sotto-oggetti `A` distinti, e la chiamata diventa **ambigua** — errore di compilazione, non un semplice calo di prestazioni. Va risolta con una chiamata qualificata (`d.B::f();`).

Verificato in `prove/07_diamond_no_virtual_ambiguous.cpp` (compilato ed eseguito).

Se **entrambe** le classi intermedie facessero l'override, non esisterebbe più un final overrider unico: il programma diventa **ill-formed alla definizione** della classe che le unisce (non al punto di chiamata!) — a meno che quella classe non fornisca il proprio override, che allora domina su entrambi.

Verificato in `prove/es_03_diamond_twist.cpp` (compilato ed eseguito).

6.4 Layout e costo in memoria

Example 6.4.1 (L'esempio Animal/Mammal/WingedAnimal/Bat)

Con ereditarietà virtuale il layout di `Bat` diventa:

```
(vpointer, Mammal, vpointer, WingedAnimal, Bat, Animal)
```

Il sotto-oggetto `Animal` finisce **in fondo** (non all'inizio): `Mammal` e `WingedAnimal` devono conservare il layout che avrebbero da soli, quindi l'unica copia condivisa di `Animal` va in una posizione che non appartiene a nessuno dei due rami. Servono **due vpointer**: ciascun ramo deve sapere "quanto dista da qui il mio `Animal`" (*vbase offset*), e questa distanza non è nota a compile time (dipende da come il sotto-oggetto sarà usato). La dimensione dell'oggetto cresce di due puntatori rispetto alla versione non virtuale, in cambio dell'unicità di `Animal` e dell'assenza di ambiguità.

Fonte: Wikipedia, Virtual inheritance.

Proposition 6.4.1 static_cast è vietato, serve dynamic_cast

Un `static_cast` da un puntatore a base virtuale verso la derivata è **vietato** (il compilatore dovrebbe sottrarre un offset che non conosce staticamente). Va usato `dynamic_cast`, che legge l'informazione dalla vtable a runtime — l'unico caso in cui è **obbligatorio** pur non essendoci incertezza sul tipo effettivo.

6.5 Metodi virtuali puri nel diamante

Con ereditarietà virtuale, un metodo virtuale puro va implementato **una sola volta** nella gerarchia: il ramo che non lo implementa eredita l'implementazione dall'altra parte del diamante, per dominanza.

Verificato in `prove/09_pure_virtual_diamond.cpp` (compilato ed eseguito).

6.6 Quando usarla

Proposition 6.6.1 Il criterio

L'ereditarietà virtuale serve quando l'ereditarietà esprime una **restrizione di un insieme** (un `Bat` *è* un `Animal`, uno solo) e non una **composizione di parti** indipendenti (nel qual caso copie separate sono accettabili).

Esempio reale nella libreria standard: `basic_istream` e `basic_ostream` derivano virtualmente da `basic_ios`, così `basic_iostream` (da cui `stringstream`, `fstream`) ha un solo stato di stream condiviso invece di due.

Chapter 7

Oltre le slide: approfondimenti da Wikipedia

La voce Wikipedia *Virtual function* elenca, in fondo alla pagina, una serie di riferimenti specifici sul tema più insidioso di questo capitolo: cosa succede chiamando una funzione virtuale dal costruttore o dal distruttore. Questo capitolo li riprende.
Fonte: Wikipedia, Virtual function.

7.1 Il corner case: una funzione virtuale pura può avere un corpo

Wrong Concept 7.1: Una funzione virtuale pura (`= 0`) non può avere nessuna implementazione

Falso, ed è un "corner case" citato esplicitamente da Wikipedia (Raymond Chen, *"C++ corner case: you can implement pure virtual functions in the base class"*): `= 0` impone solo che le derivate concrete facciano l'override, non vieta un corpo. Quel corpo è chiamabile solo per chiamata qualificata — o, come nel caso seguente, dal costruttore della classe stessa.

Fonte: Wikipedia, Virtual function.

Example 7.1.1 (Verificato: cosa succede DAVVERO chiamando una pura dal ctor)

Contrariamente a quanto ci si aspetterebbe (un crash a runtime), il comportamento reale osservato (g++ 15.2.1, compilando esattamente il codice sotto) è un **errore di link**, non un crash a runtime:

```
struct Base {
    virtual void f() = 0;
    Base() { f(); }             // chiamata dal ctor
};
struct Derivata : Base {
    void f() override { std::cout << "Derivata::f\n"; }
};
// g++: undefined reference to 'Base::f()'
```

Il motivo: dentro il costruttore di `Base`, il compilatore *sa* che il tipo dinamico dell'oggetto è esattamente `Base` (non ancora `Derivata`): la chiamata `f()` viene **devirtualizzata a compile time** in una chiamata diretta a `Base::f()`. Ma `Base::f()` non esiste (pura, senza corpo): è il **linker**, non il runtime, ad accorgersene. Fornendo un corpo a `Base::f()` (il corner case sopra), tutto funziona e quel corpo viene eseguito.

Verificato in `prove/05_pure_virtual_ctor.cpp` (compilato ed eseguito).

7.2 Il consiglio più citato: mai chiamare virtual da ctor/dtor

Tra i riferimenti in fondo alla voce Wikipedia compare l'articolo di Scott Meyers, *"Never Call Virtual Functions during Construction or Destruction"* (il capitolo 9 della terza edizione di *Effective C++*): la regola pratica — indipendentemente dal fatto che il metodo sia puro o abbia un corpo — è di non farlo mai, perché il risultato (dispatch sulla classe "corrente", non su quella finale) è quasi sempre contro-intuitivo per chi legge il codice.
Fonte: Wikipedia, Virtual function.
Verificato in `prove/04_ctor_dtor_virtual_call.cpp` *(compilato ed eseguito).*

7.3 Argomenti collegati (See also)

La voce Wikipedia elenca, tra gli argomenti collegati: *Abstract method*, *Inheritance*, *Superclass*, *Virtual inheritance* (capitolo 6), *Virtual class*, *Interface*, **Component Object Model (COM)** e *Virtual method table* (capitolo 3).
Fonte: Wikipedia, Virtual function.

Note:

Il **Component Object Model** di Microsoft è l'esempio più noto di VFT usata come formato di interoperabilità binaria *fuori* da un singolo linguaggio: un'interfaccia COM è, letteralmente, un puntatore a una VFT con un layout standardizzato — lo stesso meccanismo di questo capitolo, usato per far comunicare linguaggi diversi (C++, VB, ecc.) tramite un contratto binario comune.

Chapter 8

Overriding in Java

Questo capitolo si appoggia sulla voce Wikipedia *Method overriding* (sezione Java), sulla parte già vista dalle slide (capitolo 2) e sul file verificato `prove/17_JavaOverrideAnnotationAndCovariance.java`.
Fonte: Wikipedia, Method overriding.

8.1 Overriding automatico, e l'annotazione `@Override`

Proposition 8.1.1 In Java l'overriding è automatico

A differenza del C++, dove serve dichiarare esplicitamente `virtual`, in Java l'overriding avviene **automaticamente** ogni volta che un metodo di una sottoclasse combina lo stesso nome, gli stessi parametri e lo stesso return type (o uno covariante) di un metodo della superclasse.

Fonte: Wikipedia, Method overriding.
L'annotazione `@Override` è **opzionale ma raccomandata**: non cambia il comportamento a runtime, ma dice al compilatore "questo metodo deve essere un override di qualcosa: se non lo è, dammi errore".
Fonte: Wikipedia, Method overriding.

Example 8.1.1 (Perché `@Override` va sempre usata)

```
class Animal { public void makeSound() { System.out.println("..."); } }
class Dog extends Animal {
    @Override
    public void makeSound() { System.out.println("bau"); }

    // Un typo nel nome del metodo, SENZA @Override, compila
    // silenziosamente: diventa un OVERLOAD inutile, non un override.
    // public void makeSund() { System.out.println("bau (typo)"); }
    //
    // Con @Override, lo stesso typo da' un errore di compilazione:
    // "method does not override a method from its superclass".
}
```

Verificato in `prove/17_JavaOverrideAnnotationAndCovariance.java` (compilato ed eseguito).

8.2 Cosa non può essere overridden

`final`, `private` e `static` non possono essere overridden; una classe `final` non può essere estesa affatto.
Fonte: Wikipedia, Method overriding.
Fonte: slide ExtraLectureOnVF*, pag. 16.*

Wrong Concept 8.1: Un metodo private in una sottoclasse con lo stesso nome della base fa l'ove

No: i metodi `private` non sono ereditati né virtual (invocati con `invokespecial`, non con dispatch dinamico). Un metodo omonimo nella sottoclasse è un metodo del tutto **indipendente**, non un override — esattamente come i metodi `private` in C++ (capitolo sul pattern NVI del fascicolo d'esame).

8.3 super: chiamare l'implementazione della base

Java usa la keyword `super` per invocare esplicitamente l'implementazione della superclasse — concettualmente equivalente al `Base::f()` del C++ (che invece usa lo scope-resolution operator, non una keyword dedicata).
Fonte: Wikipedia, Method overriding.

8.4 I campi NON sono polimorfici

Wrong Concept 8.2: Se un metodo overridden dispatcha sul tipo reale, anche i campi si comport

No. I **campi** sono risolti a compile time sul tipo **statico** della variabile/riferimento, indipendentemente dal tipo reale dell'oggetto — il polimorfismo in Java riguarda *solo* i metodi.

```
class A { String x = "A"; String get() { return x; } }
class B extends A { String x = "B"; String get() { return x; } }
A a = new B();
System.out.println(a.x);       // "A": campo, risolto sul tipo statico A
System.out.println(a.get());   // "B": metodo, risolto sul tipo dinamico B
```

`B.x` non sostituisce `A.x`: lo **nasconde** (nell'oggetto esistono due campi distinti, accessibili con `super.x` o un cast).

8.5 Return type covariante

Da Java 5, un override può restituire un **sottotipo** del return type dichiarato nella base (non necessariamente lo stesso tipo esatto).
Verificato in `prove/17_JavaOverrideAnnotationAndCovariance.java` (compilato ed eseguito).

```
class Animal2 { Animal2 clone2() { return new Animal2(); } }
class Dog2 extends Animal2 {
    @Override
    Dog2 clone2() { return new Dog2(); }   // Dog2 e' un sottotipo di Animal2
}
```

8.6 La chiamata a un metodo overridabile dal costruttore

Il punto di confronto più importante con il C++ (capitolo 7): in Java il tipo dinamico di un oggetto è **già quello finale** fin dal primo istante della costruzione (a differenza del C++, dove "cresce" durante la costruzione, capitolo 3). Chiamare un metodo overridabile dal costruttore della base dispatcha quindi **correttamente** sulla sottoclasse — ma i campi di quella sottoclasse valgono ancora il **default** (anche se `final`!), perché i suoi inizializzatori girano solo *dopo* il ritorno da `super()`.
Verificato in `prove/10_VirtualFunctionsInJava.java` (compilato ed eseguito).

Note:

Regola pratica identica a quella del C++ (Scott Meyers, capitolo 7), anche se il meccanismo che la causa è diverso: **mai chiamare, dal costruttore, un metodo che una sottoclasse potrebbe overridare**, in nessuno dei due linguaggi.

Chapter 9

Overriding in Python

Questo capitolo si appoggia sulla voce Wikipedia *Method overriding* (sezione Python) e sui file verificati `prove/14–16`.
Fonte: Wikipedia, Method overriding.

9.1 Nessuna keyword, perché non serve

In Python **ogni** metodo è dinamicamente dispatchato per costruzione: il lookup di un attributo segue sempre l'MRO (*Method Resolution Order*) del tipo **reale** dell'oggetto. Non esiste una versione "non virtuale" tra cui scegliere, quindi non c'è nulla da dichiarare esplicitamente — a differenza di C++ (dove serve `virtual`) e, in misura minore, di Java (dove tutto è virtual di default, ma esiste comunque `final` per vietarlo).
Verificato in `prove/11_python_no_virtual_keyword.py` (compilato ed eseguito).

9.2 super() e l'MRO: ereditarietà multipla cooperativa

Wikipedia nota che Python usa `super(Subclass, self).method()` (o, in Python 3, la forma semplificata `super().method()`) per invocare l'implementazione della classe base.
Fonte: Wikipedia, Method overriding.

Wrong Concept 9.1: super() chiama sempre "la superclasse"

Non è la superclasse: è il **prossimo elemento nell'MRO dell'oggetto**. In una gerarchia con ereditarietà multipla, questo è ciò che garantisce che un metodo comune a più rami del diamante venga eseguito **una sola volta**, non una per ramo.

Example 9.2.1 (Il diamante cooperativo, verificato)

```
class A:
    def f(self): print("A.f")
class B(A):
    def f(self): print("B.f"); super().f()
class C(A):
    def f(self): print("C.f"); super().f()
class D(B, C):
    def f(self): print("D.f"); super().f()

print(D.__mro__)   # (D, B, C, A, object)
D().f()
```

Output verificato: `D.f B.f C.f A.f` (poi il "dopo" di ciascuno, in ordine inverso) — `A.f` viene eseguito **una sola volta**. Dentro `B.f`, con `type(self) is D`, il successivo nell'MRO dopo `B` è `C`, non `A`: è per questo che `super()` in `B` non salta direttamente ad `A`.

Verificato in `prove/14_python_mro_c3.py` (compilato ed eseguito).

9.3 Name mangling: l'equivalente di un metodo non virtuale

Python non ha `private` né un modo di impedire l'override in senso stretto, ma il **doppio underscore** iniziale (`__nome`) dà un effetto simile tramite il **name mangling**: il compilatore riscrive `self.__helper` in `self._NomeClasse__helper`, in base alla classe **dentro cui** il codice è scritto (non alla classe reale dell'oggetto a runtime).

Example 9.3.1 (Perché `B.__helper` non fa l'override di `A.__helper`)

```
class A:
    def __init__(self): self.__helper()   # diventa self._A__helper()
    def __helper(self): print("A helper")
class B(A):
    def __helper(self): print("B helper")  # diventa _B__helper: nome diverso!
B()   # stampa "A helper", non "B helper"
```

I due metodi hanno nomi mangled **diversi** (`_A__helper` e `_B__helper`): non c'è alcuna collisione, quindi nessun override. Non è un meccanismo di sicurezza: il nome mangled resta accessibile da fuori (`b._A__helper()`).

Verificato in `prove/15_python_name_mangling.py` (compilato ed eseguito).

9.4 Metodi "abstract": @abstractmethod da solo non basta

Wrong Concept 9.2: @abstractmethod impedisce di istanziare una classe che non fa l'override

Da solo, no: `@abstractmethod` si limita a impostare `__isabstractmethod__ = True` sulla funzione. Il controllo che impedisce l'istanziazione è fatto dalla **metaclasse** `ABCMeta` (che popola `__abstractmethods__` alla creazione della classe): senza ereditare da `abc.ABC` (o impostare esplicitamente quella metaclasse), l'istanziazione riesce comunque, senza alcun errore.

Verificato in `prove/16_python_abc_not_enforced.py` (compilato ed eseguito).

9.5 Il costruttore e il dispatch

Come in Java (capitolo 8), il tipo dinamico di `self` è già quello finale dal primo istante della costruzione: una chiamata a un metodo overridabile da `__init__` dispatcha correttamente sulla sottoclasse. Vale però lo stesso avvertimento: se quel metodo legge un attributo che l'`__init__` della sottoclasse non ha ancora assegnato, il risultato può essere sbagliato (o, in Python, un `AttributeError` se l'attributo non esiste affatto prima della prima assegnazione).

Verificato in `prove/11_python_no_virtual_keyword.py` (compilato ed eseguito).

Chapter 10

Polimorfismo manuale in C

Il C non ha funzioni virtuali, classi, né alcuna forma di overriding nel linguaggio: la voce Wikipedia *Method overriding* non ha nemmeno una sezione dedicata al C, per questo motivo. Questo capitolo traduce **letteralmente a mano** il meccanismo VFT/VFTP del capitolo 3, verificando col compilatore reale sia il funzionamento sia i modi in cui si rompe.

10.1 La traduzione letterale di VFT e VFTP

Example 10.1.1 (Dispatch scritto a mano, verificato)

```c
/* la VFT: una tabella di puntatori a funzione, UNA per classe */
typedef struct {
    void   (*draw)(const void *self);
    double (*area)(const void *self);
} ShapeVT;

/* la "classe base": porta solo il VFTP */
typedef struct { const ShapeVT *vptr; } Shape;

/* la "classe derivata": incorpora Shape come PRIMO membro */
typedef struct { Shape base; double r; } Circle;

static double circle_area(const void *self) {
    const Circle *c = (const Circle *)self;
    return 3.14159265 * c->r * c->r;
}
static const ShapeVT CIRCLE_VT = { circle_draw, circle_area };

Circle *circle_new(double r) {
    Circle *c = malloc(sizeof *c);
    c->base.vptr = &CIRCLE_VT;   /* il "costruttore" installa il vptr a mano */
    c->r = r;
    return c;
}
/* dispatch "virtuale": gli stessi 5 passi della teoria (cap. 3) */
double shape_area(const Shape *s) { return s->vptr->area(s); }
```

Un ciclo su un array di `Shape*` misti (`Circle`, `Rectangle`) chiama `shape_area` **senza nessun** `if`/`switch` **sul tipo**: esattamente il late binding della teoria, scritto a mano.

Verificato in `prove/12_manual_vtable_in_c.c` *(compilato ed eseguito).*

Proposition 10.1.1 Il primo membro deve essere la base

Lo standard C garantisce l'assenza di padding all'inizio di una struct: quindi `(Shape*)circle == &circle->base`, e la conversione derivata→base è un no-op, senza bisogno di alcun aggiustamento del

puntatore (nessun *thunk*, a differenza dell'ereditarietà multipla C++).

10.2 Chi inizializza il vptr: qui nessuno, se non tu

Wrong Concept 10.1: Se dimentico di inizializzare il vptr, il programma comunque funziona un p

No: senza quella riga, il vptr è memoria non inizializzata (`malloc` non azzera la memoria). Chiamare una funzione attraverso un vptr indeterminato è undefined behavior, e nella pratica **crasha davvero** (verificato: `Segmentation fault`).

In C++, questa riga di inizializzazione il programmatore **non la scrive mai a mano**: il compilatore la genera automaticamente in ogni costruttore, per ogni livello della gerarchia (capitolo 3). In C, dimenticarla è un errore reale e frequente.
Verificato in `prove/13_c_no_forgotten_vptr.c` (compilato ed eseguito).

10.3 Esempi reali: non è un esercizio accademico

Questo identico schema (struct con un puntatore a una tabella di puntatori a funzione) è usato da progetti C reali su larga scala:

- il **VFS** (Virtual File System) del kernel Linux, dove `struct file_operations` è, letteralmente, una VFT: ogni filesystem (ext4, procfs, un driver...) fornisce la propria tabella, e il kernel dispatcha tramite quella tabella senza sapere di che filesystem si tratti;
- **GObject** (la base di GTK): implementa un intero sistema di classi con ereditarietà single/multipla (via interfacce) e dispatch virtuale, tutto scritto a mano sopra il C puro, esattamente con questo schema.

Note:

Da distinguere dallo schema di `qsort(arr, n, sizeof(int), cmp)`: lì il puntatore a funzione (`cmp`) è passato **per parametro alla singola chiamata**, non associato al *tipo* degli oggetti ordinati. È il pattern *Strategy*, non l'overriding: non esiste alcuna "classe" che eredita quel comportamento.

Chapter 11

Riferimenti e link aggiuntivi

Come indicato dal docente: Wikipedia va bene come riferimento, ma è meglio eseguire gli esempi ed approfondire con i link che Wikipedia stessa elenca in fondo alla pagina, più il libro *Thinking in C++*.

11.1 Dalla voce "Virtual function"

See also: Abstract method, Inheritance, Superclass, Virtual inheritance, Virtual class, Interface, Component Object Model, Virtual method table.
Riferimenti specifici sul tema ctor/dtor (il più utile per questo capitolo): Raymond Chen, *"What is __purecall?"*; Scott Meyers, *"Never Call Virtual Functions during Construction or Destruction"*; Raymond Chen, *"C++ corner case: you can implement pure virtual functions in the base class"*; S.G. Ganesh, *"Joy of Programming: Calling Virtual Functions from Constructors"*.

11.2 Dalla voce "Virtual inheritance"

Milea, *"Solving the Diamond Problem with Virtual Inheritance"* (Cprogramming.com); McArdell, *"C++/What is virtual inheritance?"*.

11.3 Dalla voce "Method overriding"

See also: Implementation inheritance, Inheritance semantics, Method overloading, Polymorphism in object-oriented programming, Template method pattern, Virtual inheritance.

11.4 Thinking in C++ (Bruce Eckel) — libero, due volumi

- Vol. 2, cap. 9 — *Multiple Inheritance*, sezione *Virtual base classes*: https://web.mit.edu/merolish/ticpp/TicV2.html#_Toc53985825
- Vol. 2, cap. 5 — *Templates in Depth* (per il capitolo sui template: non-type/template template parameter, SFINAE, CRTP, metaprogramming): https://web.mit.edu/merolish/ticpp/TicV2.html#_Toc53985706

Chapter 12

Fonti

- Slide del corso: *Virtual Functions and Late Bindings* (`20250922.ExtraLectureOnVF.pdf`), 45 pagine — fonte primaria dei capitoli 1–5.
- Wikipedia, *Virtual function*, https://en.wikipedia.org/wiki/Virtual_function — fonte primaria del capitolo 7.
- Wikipedia, *Virtual inheritance*, https://en.wikipedia.org/wiki/Virtual_inheritance — fonte primaria del capitolo 6.
- Wikipedia, *Method overriding*, https://en.wikipedia.org/wiki/Method_overriding — fonte primaria dei capitoli 8 e 9.
- Bruce Eckel, *Thinking in C++*, Vol. 2 (*Practical Programming*), cap. 9 — fonte di supporto per il capitolo 6.
- Cartella `prove/` (questo stesso computer): ogni esempio citato con *Verificato in `prove/` (compilato ed eseguito).* è stato compilato ed eseguito davvero, non solo descritto.